



Facultad de Ingeniería - Udelar

Práctico 3

Memoria compartida y programación a nivel de warp

Belén Olivera

Nicolás Pereira

Manuel Roca

Ejercicio 1 - Memoria compartida

Resolvemos el ejercicio experimentando con una matriz cuadrada con $N = 2^{14}$. Los bloques usados son de 32×32 .

Parte 1.1

En esta parte utilizamos memoria compartida definiendo tiles de 32×32 elementos. Dentro de cada tile, cada hilo accede a su posición correspondiente usando sus coordenadas dentro del bloque.

Luego cada hilo calcula el índice global del elemento que tiene que cargar en la tile:

$$x = blockIdx.x * blockDim.x + threadIdx.x$$

$$y = blockIdx.y * blockDim.y + threadIdx.y$$

y carga el elemento correspondiente desde memoria global en la tile:

$$tile[threadIdx.y][threadIdx.x] = matriz_in[y * n + x]$$

Esta lectura es coalesced, los hilos consecutivos acceden a posiciones contiguas de memoria global (respetando el almacenamiento row-major order de C).

Luego se sincroniza con `__syncthreads()` para asegurarnos que todos los hilos hayan terminado de cargar su elemento en la tile.

Después cada hilo invierte sus coordenadas globales:

$$x = blockIdx.y * blockDim.y + threadIdx.x$$

$$y = blockIdx.x * blockDim.x + threadIdx.y$$

e invierten el orden en el cual acceden a la tile:

$$tile[threadIdx.x][threadIdx.y]$$

logrando así transponer los elementos. En esta escritura los hilos escriben posiciones contiguas en memoria global, ya que la coordenada x sigue variando consecutivamente entre hilos consecutivos.

De esta forma se logra leer coalesced desde memoria global, utilizar la memoria compartida para realizar los accesos no coalesced y luego escribir de forma coalesced en memoria global. Evitamos los accesos no coalesced en memoria global que vimos en el Práctico 2.

Para ver el beneficio, comparamos el tiempo de ejecución de esta versión con la del práctico anterior que solo usa memoria global. Usamos en ambos casos una matriz cuadrada con $2^{14} \times 2^{14}$ elementos. En ambos casos usamos tamaño de bloque 32×32 .

Tamaño del arreglo	Tiempo prom. (ms)	Desv. Estándar (ms)
Tiling	7.769318	0.044346
Memoria global	12.675771	1.764550

Cuadro 1: Comparación de tiempos de ejecución de ambas implementaciones.

Los tiempos de ejecución son mejores al usar la memoria compartida. De todos modos, aun se podrian mejorar resolviendo los conflictos de banco que exploramos en la Parte 1.2, que perjudican el uso de memoria compartida.

Parte 1.2

La memoria compartida esta dividida en 32 módulos de igual tamaño, los bancos. Cada banco puede atender una única solicitud por ciclo. Múltiples lecturas a un mismo elemento en un banco no generan conflicto, pero si múltiples hilos intentan acceder a distintas palabras de memoria que mapean al mismo banco, se produce un conflicto de bancos. En este caso los accesos se serializan penalizando fuertemente el desempeño.

Al evaluar la lectura transpuesta `tile[threadIdx.x][threadIdx.y]`, para un warp determinado, la variable `threadIdx.y` se mantiene constante, mientras que `threadIdx.x` varía de 0 a 31. La dirección de memoria lineal (stride) a la que accede el hilo i (siendo $i = \text{threadIdx.x}$) es :

$$\text{Dirección base} + i \times 32 + \text{threadIdx.y}$$

Para determinar a que banco de memoria físico corresponde esta dirección, se calcula el módulo 32 del desplazamiento. Dado que `threadIdx.y` representa una constante C para todo el *warp*, quedaría:

$$(i \times 32 + C) \pmod{32} = C$$

Y por lo tanto todos los accesos de lectura sobre el tile caen en el mismo banco. Esto causa el peor escenario de conflicto de banco posible, un 32-way bank conflict donde se serializan los 32 accesos de lectura. Al obligar a la memoria compartida a serializar 32 transacciones consecutivas, se anula todo el beneficio de usarla como caché, lo que explica por qué esta versión (0.548 ms) resultó más lenta que la ejecución puramente sobre memoria global (0.211 ms).

Al agregar el padding, tenemos 33 elementos por fila, entonces la distancia en memoria compartida de cada elemento de una misma columna ya no es múltiplo de 32. El nuevo cálculo del banco para los hilos del warp durante la lectura pasa a ser:

$$(i \times 33 + C) \pmod{32} = (i + C) \pmod{32}$$

Como cada hilo i varía de 0 a 31, se asocia un banco $(i + C)$ distinto para cada uno. De esta forma se elimina totalmente el problema de conflictos de banco, y todos los hilos del warp pueden leer de memoria compartida en paralelo.

Para implementar esto agregamos una columna adicional en la tile de memoria compartida, por lo que la definimos de esta forma: `tile[32][33]`

Ejecutamos el kernel y obtenemos los siguientes resultados:

Tamaño del arreglo	Tiempo prom. (ms)	Desv. Estándar (ms)
Tiling con padding	5.246503	0.003041

Cuadro 2: Comparación de tiempos de ejecución de ambas implementaciones.

Como era de esperarse, se observa una mejora considerable respecto a la version de tiling sin padding.

Si bien incluir esta columna dummy de padding hace que definamos espacio en memoria compartida que no usamos, el beneficio es enorme al lograr resolver los conflictos. Esto se evidenció en los tiempos de ejecución obtenidos.

Ejercicio 2 - Warp shuffle

El objetivo del ejercicio es implementar un kernel que para arreglos de tamaño múltiplo de 32, para cada segmento de 32 elementos consecutivos reemplace todos los elementos negativos por la suma de estos y el valor del máximo positivo del segmento.

Por simplicidad, decidimos usar bloques de 32 hilos para que cada bloque sea exactamente un warp. En cuanto a las pruebas, ejecutamos con arreglos de 2^{22} y 2^{28} elementos para comparar un escenario chico frente a uno gigante. Estimamos que con tamaños como 2^{22} la GPU queda muy subutilizada, por lo que esperábamos notar el verdadero impacto del paralelismo y del uso de la memoria recién al probar con 2^{28} , que es donde realmente logramos saturar el hardware de la tarjeta.

Los valores de estos arreglos son enteros generados aleatoriamente entre -100 y 100. Todos los resultados presentados corresponden al promedio de 10 ejecuciones, medidos con la herramienta Nsight Systems. Utilizamos la tarjeta NVIDIA GTX 2080 Ti.

Parte 2.1

En esta parte resolvemos el problema usando solo memoria compartida, sin primitivas de warp.

Abordamos este problema de dos maneras distintas, una utilizando una solución secuencial y otra utilizando la técnica de reducción vista en teórico (pero sin usar primitivas warp shuffle). La idea de implementar estas dos alternativas fue usar la versión secuencial como punto de comparación. De esta forma, podíamos contrastar los tiempos y comprobar en la práctica qué tanta mejora obteníamos al paralelizar el cálculo usando el patrón de reducción.

Para la versión lineal comenzamos definiendo los siguientes elementos en memoria compartida: una tile de 32 elementos, una variable suma_neg y una variable max_pos.

Luego, los hilos cargan su elemento correspondiente en la tile. A continuación, hacemos un `__syncthreads()` fundamental para garantizar que todos los datos estén cargados y visibles. Luego el primer hilo del bloque (hilo 0) recorre todos los elementos del segmento para determinar la suma de negativos y el máximo positivo, escribiendo estos resultados en las variables compartidas suma_neg y max_pos. Seguido a esto hacemos un segundo `__syncthreads()` para asegurarnos de que el resto de los hilos vean los resultados antes de continuar y finalmente cada hilo chequea si debe cambiar su valor por el resultado de la operación o no, haciéndolo si corresponde.

Si bien el hilo 0 va a tener que hacer lecturas secuenciales sobre todos los elementos del segmento, al hacerse directamente sobre memoria compartida esto se puede hacer rápido, por lo que es una idea factible ya que los segmentos son de pocos elementos (32).

Los tiempos de ejecución obtenidos fueron los siguientes:

Tamaño del arreglo	Tiempo prom. (ms)	Desv. Estándar (ms)
2^{22}	0.305662	0.002317
2^{28}	13.386323	0.020856

Cuadro 3: Tiempos de ejecución del kernel secuencial.

Por otro lado, para la versión con reducción hicimos lo siguiente. Definimos dos tiles en memoria compartida de 32 elementos cada uno: tile_negativos[32] y tile_positivos[32]. La idea fue calcular en la misma reducción la suma de negativos y el máximo positivo, por eso definimos estos dos tiles separados.

Al principio cada hilo chequea si su valor es positivo o negativo, y escribe el valor correspondiente en las tiles. Luego hacemos `__syncthreads()` y la reducción siguiendo lo visto en teórico, de

modo que la suma de negativos queda almacenada en `tile_negativos[0]` y el máximo positivo en `tile_positivos[0]`. Finalmente cada hilo chequea si debe reemplazar su valor y puede obtener el resultado desde las tiles.

Los tiempos de ejecución fueron los siguientes:

Tamaño del arreglo	Tiempo prom. (ms)	Desv. Estándar (ms)
2^{22}	0.210612	0.000913
2^{28}	10.860292	1.524774

Cuadro 4: Tiempos de ejecución del kernel con reducción.

Viendo los resultados, la versión con reducción efectivamente baja los tiempos (pasamos de 13.38 ms en la secuencial a 10.86 ms). Esto nos comprueba en la práctica que paralelizar sirve, pero la verdad es que esperábamos una mejora más drástica al aplicar el patrón de reducción. Discutiendo sobre cómo funciona la arquitectura de la GPU por debajo, llegamos a la conclusión de que esto se debe a dos motivos principales. Primero, este kernel está fuertemente limitado por la memoria. La inmensa mayoría del tiempo de las dos versiones se nos va en la lectura inicial: traer los 2^{28} elementos desde la memoria global (que tiene una latencia enorme) para guardarlos en la tile compartida. El patrón de reducción nos acelera las cuentas que hacemos después, pero el cuello de botella de acceder a la memoria global nos sigue frenando casi igual. En segundo lugar, el tamaño del segmento que estamos procesando es mínimo (apenas 32 elementos). En la versión con reducción, el hecho de tener que meter múltiples barreras de sincronización (`__syncthreads()`) en cada paso del árbol termina penalizando el rendimiento. Ese costo extra de pausar a los hilos a cada rato nos termina comiendo la ventaja del paralelismo, sobre todo si lo comparamos con hacer un simple `for` de 32 vueltas en la versión secuencial usando la memoria compartida. Nos da la sensación de que la ventaja de usar una reducción sería muchísimo más evidente si el costo de la operación matemática fuera mayor o si el tamaño del segmento fuera mucho más grande.

Observando las desviaciones estándar, vemos que en el caso del arreglo grande, es mucho menor en la versión secuencial. Ese tiempo no es fijo, sino que depende de la carga dinámica que tenga el hardware en ese milisegundo exacto

Parte 2.2

En esta parte resolvemos el problema utilizando primitivas de warp.

Utilizamos la primitiva `_shfl_down_sync` al hacer la reducción para obtener la suma de negativos y el máximo positivo del arreglo. Luego usamos `__shfl_sync` para difundir el resultado, que se encontraría en lane 0 tras la reducción. Finalmente utilizamos `__ballot_sync` para obtener una máscara que indique los hilos con valores negativos del segmento.

La máscara utilizada en los `__shfl_down_sync` de la reducción fue `0xFFFFFFFF`, indicando que todos los hilos participan. Esto es porque la máscara define que hilos deben estar listos para ejecutar la primitiva, y de no incluirlos pueden quedar valores indeterminados en otras lanes, lo que introduciría problemas.

De todos modos entendemos que al usar esa máscara estamos serializando las operaciones de suma negativos y máximo positivo. Al final de la sección explicamos otras aproximaciones que pensamos y probamos.

Los tiempos de ejecución obtenidos fueron los siguientes:

Tamaño del arreglo	Tiempo prom. (ms)	Desv. Estándar (ms)
2^{22}	0.142256	0.0005668
2^{28}	7.851942	1.152422

Cuadro 5: Tiempos de ejecución del kernel con reducción y primitivas warp shuffle.

La mejora de rendimiento respecto a los resultados de la Parte 2.1 es considerable. Esto evidencia el beneficio de usar primitivas a nivel de warp para intercambiar los valores entre hilos directamente desde sus registros.

Exploramos otras alternativas para evitar que los calculos de suma de negativos y máximo positivo se serialicen.

Pensamos la posibilidad de utilizar la primitiva indicada en la letra (`__ballot_sync`) para obtener la mascara de las lanes positivas y negativas, y luego poder realizar una reduccion para cada conjunto de lanes en paralelo. El problema que encontramos con esta aproximacion fue que no hay una forma trivial de realizar una reducción con un conjunto arbitrario de lanes no consecutivos y se requieren primitivas no vistas en el curso como `__popc` o `__ffs`. Al intentar utilizar una reducción normal e ignorar las lanes que no corresponden al conjunto se llega al problema discutido en el foro (resultados incorrectos), de este ultimo ejemplo se incluye el codigo probado.

Explorando la documentación vimos que una técnica posible era reagrupar los valores de los hilos en las lanes, dejando los valores negativos continuos al principio. Esta opción permite conocer la cantidad de lanes con valores negativos y positivos, y así saber de antemano que hilos hay que revisar para cada operación. No experimentamos con esta vía porque necesitabamos usar varias primitivas que no vimos en clase, y requería un esfuerzo mucho mayor al de las otras versiones. De todos modos lo mencionamos porque puede ser una alternativa interesante de explorar a futuro.

Anexo

La GPU utilizada para las pruebas fue una NVIDIA GTX 2080 Ti.

Para cada ejercicio realizamos 2 tipos de pruebas. Una prueba de correctitud de la operación sobre entradas pequeñas para verificar manualmente, y pruebas de performance sobre entradas grandes.

En el Ejercicio 1 se utilizó una matriz pequeña para verificar la correctitud de la trasposición y una matriz cuadrada de tamaño $N = 2^{14}$ para medir tiempos de ejecución.

En el Ejercicio 2 se utilizaron arreglos pequeños de 64 elementos para verificar la correctitud de las operaciones, y arreglos de tamaño $N = 2^{28}$ para las mediciones de performance.

Además, en la Parte 2.2 se realizaron pruebas adicionales con las máscaras obtenidas con la primitiva `__ballot_sync`. Se incluye una ejecución de esto en el script de pruebas.

Referencias

- [1] Facultad de Ingeniería. *Diapositivas del curso: Programación de Propósito General en GPUs (GPGPU)*. Facultad de Ingeniería, Universidad de la República (UdelaR), 2026.
- [2] NVIDIA Developer Forums. *What does mask mean in warp shuffle functions (__shfl_sync)?*
Disponible en: <https://forums.developer.nvidia.com/t/67697>.